

Time Series Modelling for Share Price Prediction

3.1 Introduction

3.1.1 Scenario Summary

The scenario focuses on supporting a financial investment firm that aims to improve its decision-making by forecasting future share price movements. To achieve this, time series analysis and modelling are used to identify underlying patterns, trends, and potential future behaviour in stock prices. For this task, the EEA company was selected from the provided dataset and focus was given to a single price variable (such as the open, close, high, or low price). Using R, the objective is to explore the historical data, assess its time-dependent structure, and develop an appropriate forecasting model that can generate reliable short-term predictions to assist investment strategies.

3.1.2 Initial Data Exploration Findings

An initial examination of the dataset shows that it contains **11,835** daily observations US Bancorp with seven key variables: Date, Open, High, Low, Close, Adjusted Close, and Volume. All price-related fields are stored as numeric values with seemingly no missing entries, and the Date column is complete and consistently formatted. At a glance, there is an indication that the stock has traded within a wide historical range.

Next, we went on to install and load all necessary packages. The “rugarch” package helped in salvaging residuals autocorrelations as will be seen earlier.

```
# ----- Time Series Analysis on USB Stock -----  
  
# Installing and loading necessary packages  
install.packages("TTR")  
install.packages("forecast")  
install.packages('xts')  
install.packages("quantmod")  
install.packages('highcharter')  
install.packages('tsibble')  
install.packages("rugarch")  
  
# Loading the packages  
library(TTR)  
library(forecast)  
library(xts)  
library(dplyr)  
library(ggplot2)  
library(tsibble)  
library(highcharter)  
library(quantmod)  
library(rugarch)
```

The dataset was read into a dataframe next.

```
usb <- read.csv('USB.csv')
```

```
> head(usb)
```

	Date	Open	High	Low	Close	Adj.Close	Volume
1	1973-05-03	0.4611111	0.4805556	0.4611111	0.4611111	7.766482e-05	7200
2	1973-05-04	0.4694445	0.4805556	0.4694445	0.4694445	7.906845e-05	10800
3	1973-05-07	0.4694445	0.4805556	0.4611111	0.4611111	7.766482e-05	43200
4	1973-05-08	0.4694445	0.4805556	0.4694445	0.4694445	7.906845e-05	90000
5	1973-05-09	0.4694445	0.4888889	0.4694445	0.4694445	7.906845e-05	79200
6	1973-05-10	0.4694445	0.4888889	0.4694445	0.4694445	7.906845e-05	10800

```
> tail(usb)
```

	Date	Open	High	Low	Close	Adj.Close	Volume
11829	2020-03-25	31.40	33.74	29.99	32.03	31.65391	17018600
11830	2020-03-26	32.38	36.72	32.23	36.18	35.75518	16014300
11831	2020-03-27	34.46	37.19	34.17	35.77	35.35000	11470300
11832	2020-03-30	35.00	36.10	34.01	35.87	35.87000	9825600
11833	2020-03-31	35.44	36.37	33.92	34.45	34.45000	11034500
11834	2020-04-01	32.53	33.00	31.26	31.93	31.93000	11203900

To a look at the timeseries data, we cast the date column to DateTime and also and converted the dataframe to an xts timeseries object.

```
# cast date column to datetime object
usb$Date <- as.Date(usb$Date)
```

```
# convert to xts object & remove not needed columns
usb_ts <- xts(usb_ts[, c("Open", "High", "Low", "Close")],
             order.by = usb$Date)
```

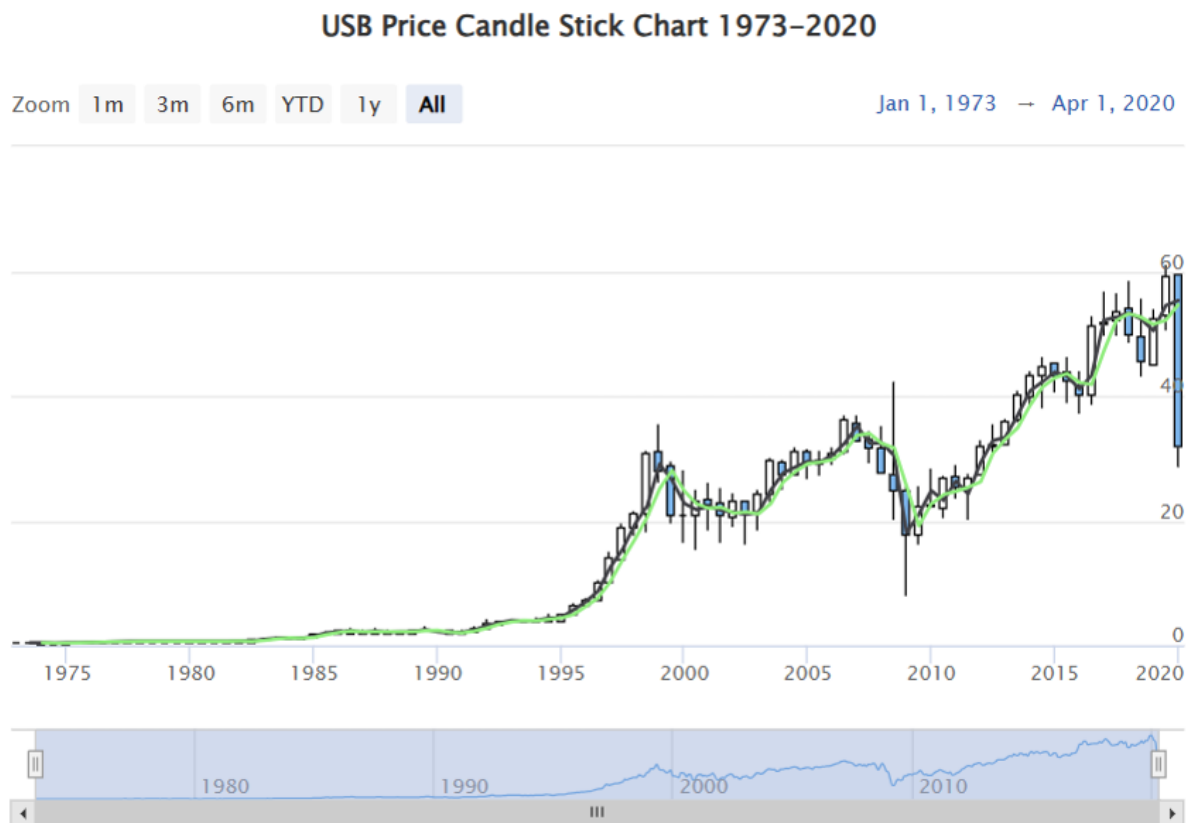
Before visualizing using chartSeries available with quantmod.

```
# visualize static chart
chartSeries(usb_ts$Close, name = "USB Price 2014-2019")
```



We also created an interactive plot using OHLC values:

```
# visualization of candlesticks with SMA
highchart(type="stock") %>%
  hc_add_series(usb_ts) %>%
  hc_add_series(SMA(na.omit(Cl(usb_ts)),n=50),name="SMA(50)") %>%
  hc_add_series(SMA(na.omit(Cl(usb_ts)),n=200),name="SMA(200)") %>%
  hc_title(text="<b>USB Price Candle Stick Chart 1973-2020</b>")
```



The webpage visual can also be accessed using this link [Interactive Chart](#)

3.2 Implementation

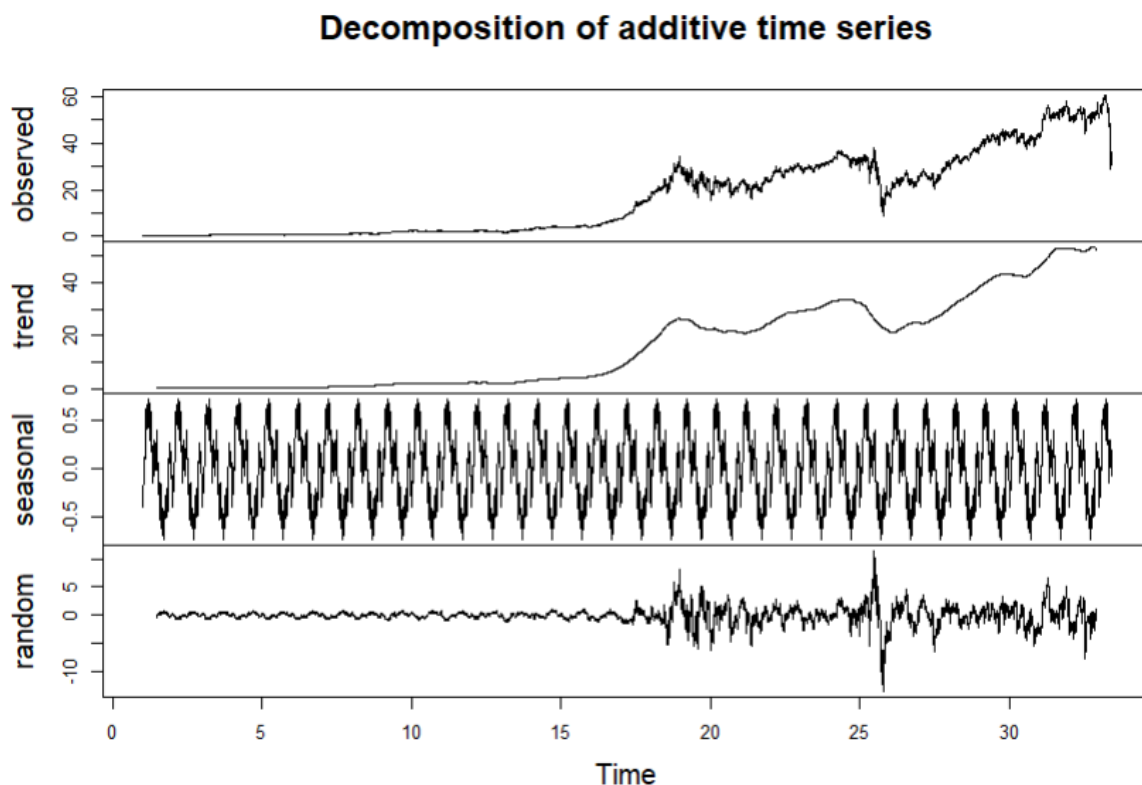
3.2.1 Decomposition

Decomposition of the different components of the series was then taken on:

```
# Decomposing components
close_ts <- ts(usb_ts$Close, frequency = 365)

decomp <- decompose(close_ts)

plot(decomp) # no seasonality
```



At a glance, one can easily confirm that there is no seasonality in the timeseries.

3.2.2 HoltWinters Smoothing and Forecasting

First, was the smoothing. Gamma was set to FALSE to disable only seasonal smoothing since there clear trend.

```
# forecast
usbseriesforecasts <- HoltWinters(close_ts, gamma=FALSE)
usbseriesforecasts

usbseriesforecasts$SSE

plot(usbseriesforecasts)

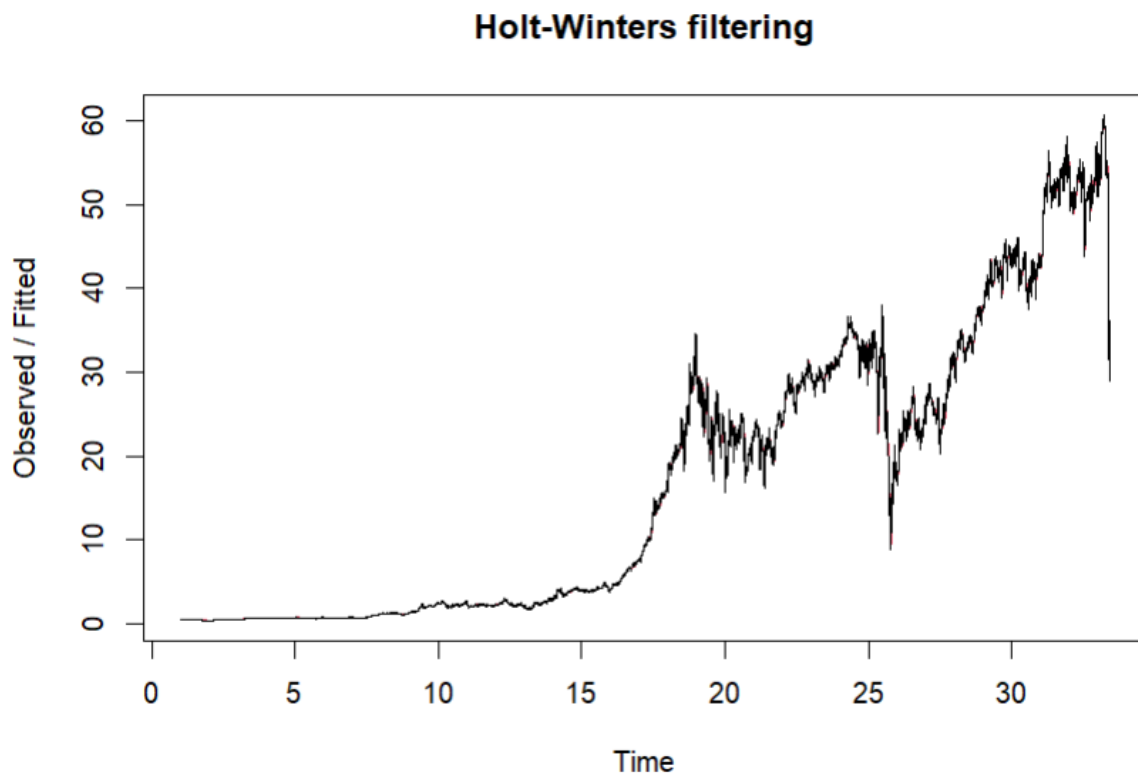
> usbseriesforecasts
Holt-Winters exponential smoothing with trend and without seasonal component.

Call:
HoltWinters(x = close_ts, gamma = FALSE)

Smoothing parameters:
alpha: 0.942466
beta : 0
gamma: FALSE

Coefficients:
      [,1]
a 32.080177892
b  0.008333355
> |

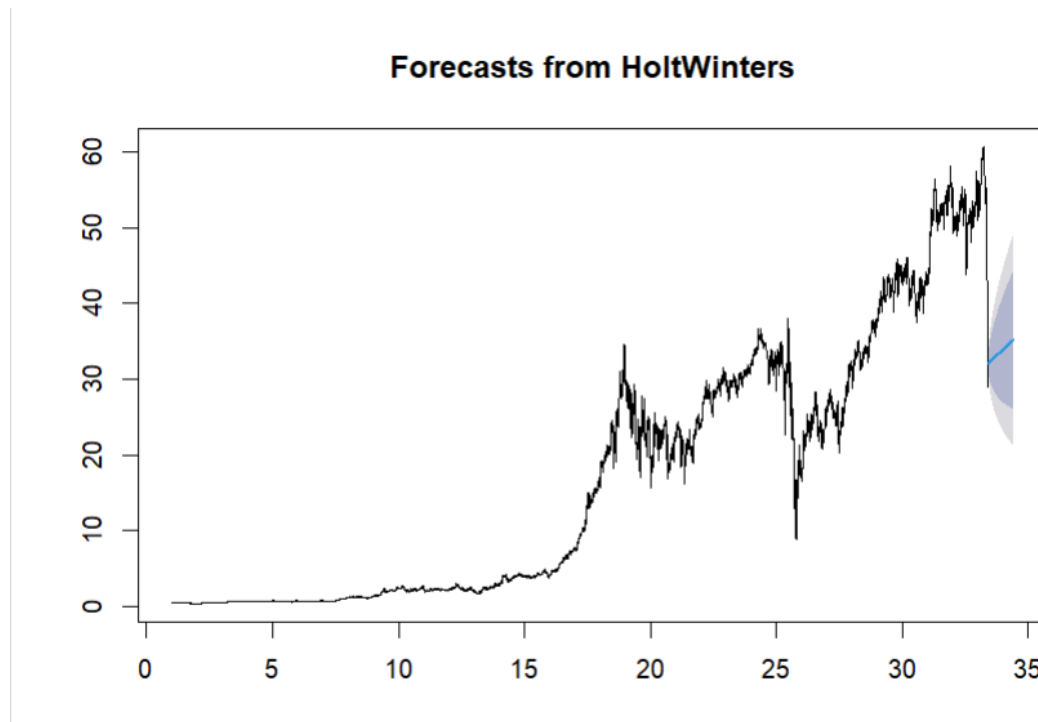
> usbseriesforecasts$SSE
[1] 1837.88
> |
```



And the forecasting:

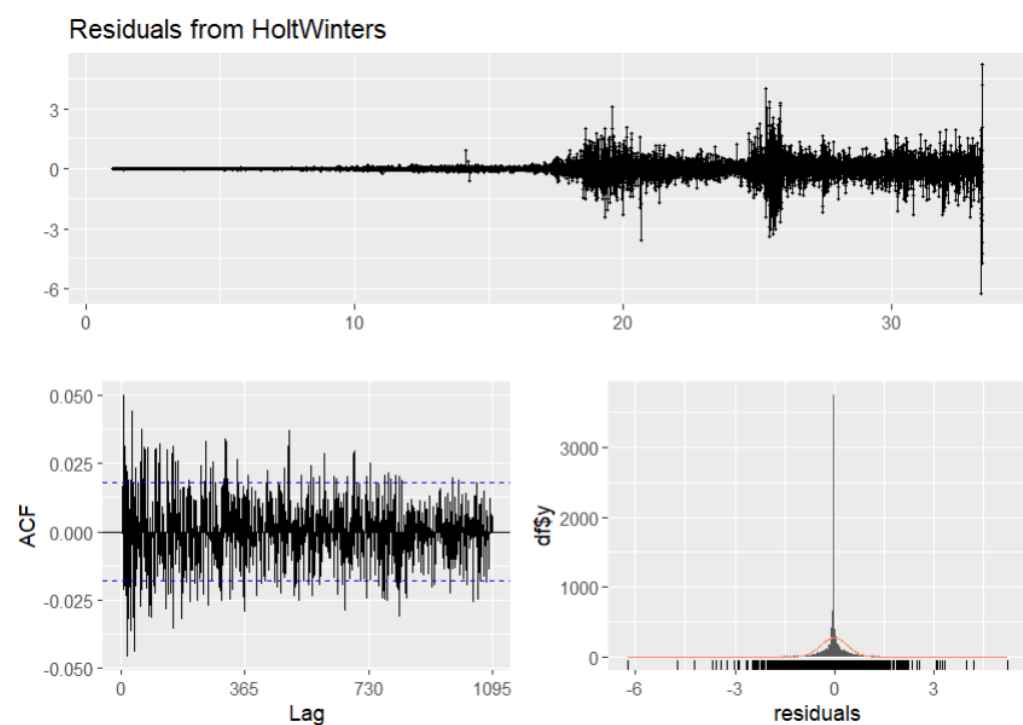
```
usbseriesforecasts2 <- forecast(usbseriesforecasts, h=365)
plot(usbseriesforecasts2)
```

Forecasting was done for a year. This was the result:

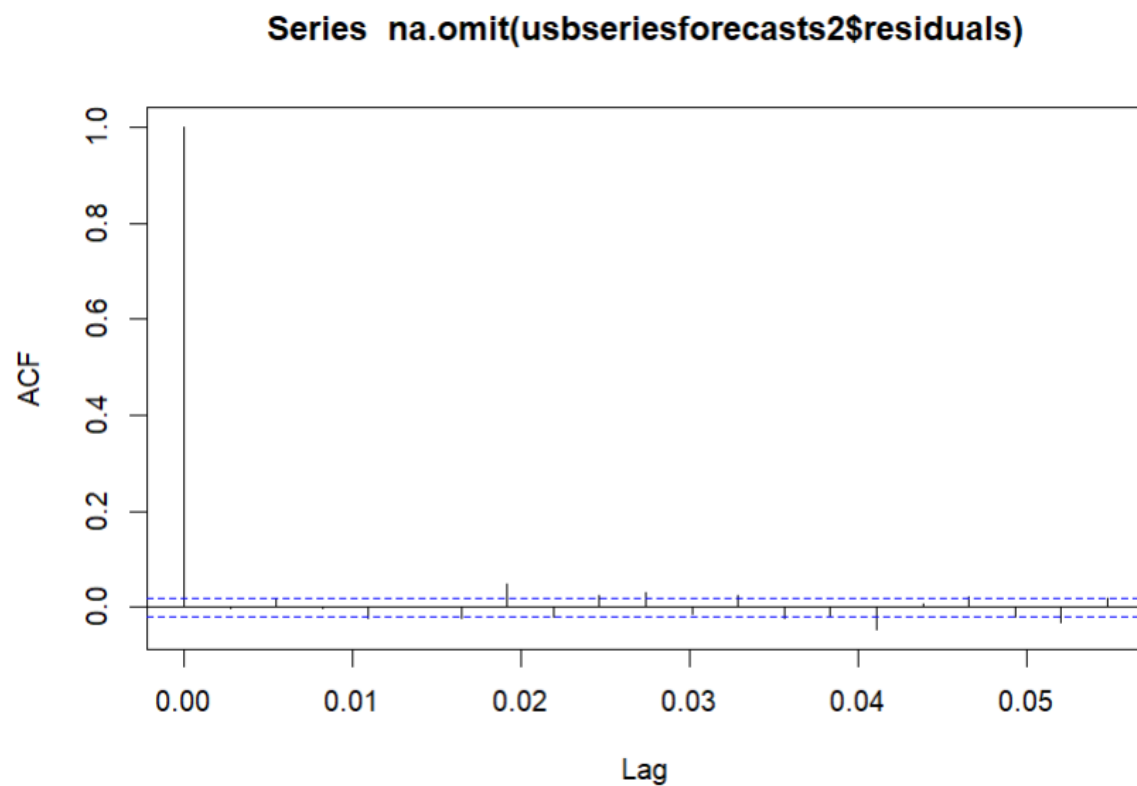


We used the checkresiduals method available with the forecast package to check assumptions:

```
checkresiduals(usbseriesforecasts2)
```



Although the residual histogram distribution was satisfactory, the ACF plot highlighted strong correlations. This was honed in on for confirmation:



```
> checkresiduals(usbseriesforecasts2)
```

Ljung-Box test

```
data: Residuals from Holtwinters  
Q* = 1582.1, df = 730, p-value < 2.2e-16
```

```
Model df: 0. Total lags used: 730
```

The model could not be relied on for accuracy as p-value is far below 0.05.

3.2.3 ARIMA Modelling

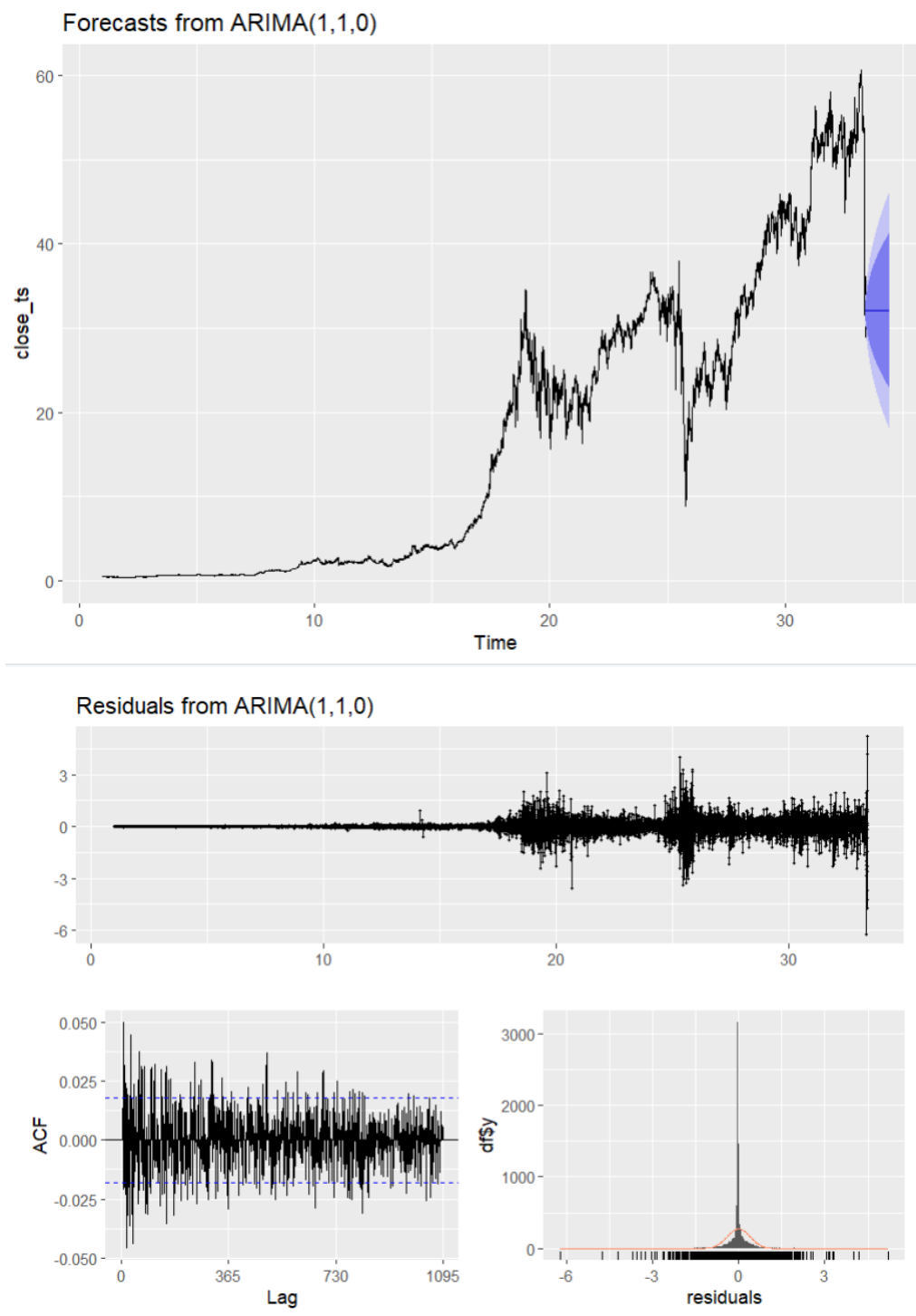
Same was true when we tried modelling with `auto.arima`.

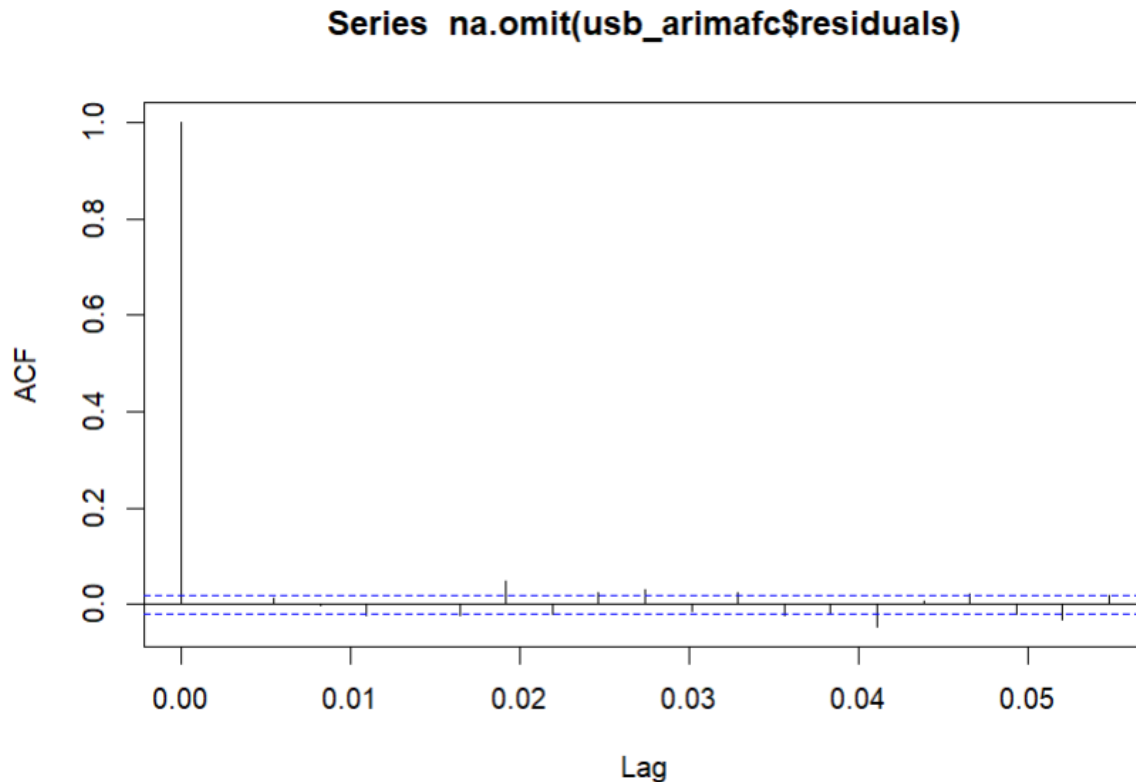

```
# Arima modelling
usb_arima <- auto.arima(close_ts, seasonal=FALSE)

# Forecast 365 days
usb_arimafc <- forecast(usb_arima, h=365)

autoplot(usb_arimafc)

checkresiduals(usb_arimafc)
```





```
> checkresiduals(usb_arimaafc)
```

```
      Ljung-Box test
```

```
data: Residuals from ARIMA(1,1,0)
Q* = 1581.5, df = 729, p-value < 2.2e-16
```

```
Model df: 1. Total lags used: 730
```

```
> |
```

Again p-value is far below 0.05 and so the model can't be relied on for even near accurate forecasting.

3.2.4 ARIMA + GARCH Modelling

Research recommended GARCH for remedying the situation. ARIMA and GARCH models are so often combined in cases like this. An ARIMA model estimates the conditional mean, where subsequently a GARCH model estimates the conditional variance present in the residuals of the ARIMA estimation (Dierckx, 2020). ARIMA models are best suited for stationary time series data, while GARCH models are best suited for financial time series data that exhibit volatility clustering (Navaei, 2023).

Close returns were used for estimating the arima mean.

```
logret <- diff(log(close_ts))
logret <- na.omit(logret)

arima_mean <- auto.arima(logret)
arima_mean
```

The order (p,q) of the ARIMA model was extracted and the parameters for the GARCH model set:

```
# Extract ARMA orders from ARIMA model
ord <- arimaorder(arima_mean)

ar_order <- ord[1] # p
ma_order <- ord[3] # q

# specify GARCH parameters
spec <- ugarchspec(
  variance.model = list(model="sGARCH", garchOrder=c(1,1)),
  mean.model = list(armaOrder=c(ar_order, ma_order), include.mean=TRUE),
  distribution.model = "norm"
)
```

After which the ARIMA-GARCH model was fitted and assumptions for residuals tested.

```
# Fit GARCH model
garch_fit <- ugarchfit(spec, logret)
garch_fit

plot(garch_fit, which = 1) # standardized residuals
plot(garch_fit, which = 2) # ACF of residuals
plot(garch_fit, which = 3) # squared residuals
```

```
*-----*
*          GARCH Model Fit          *
*-----*
```

Conditional Variance Dynamics

```
-----
GARCH Model      : sGARCH(1,1)
Mean Model       : ARFIMA(1,0,1)
Distribution      : norm
```

Optimal Parameters

```
-----
      Estimate Std. Error t value Pr(>|t|)
mu      0.000603   0.000100   6.0434 0.000000
ar1     -0.718524   0.215009  -3.3418 0.000832
ma1      0.706798   0.218814   3.2301 0.001237
omega    0.000004   0.000000   9.3218 0.000000
alpha1   0.093805   0.003542  26.4816 0.000000
beta1    0.892446   0.002962 301.2629 0.000000
```

Robust Standard Errors:

	Estimate	Std. Error	t value	Pr(> t)
mu	0.000603	0.000226	2.6665	0.007665
ar1	-0.718524	0.185838	-3.8664	0.000110
ma1	0.706798	0.189732	3.7252	0.000195
omega	0.000004	0.000002	1.8080	0.070608
alpha1	0.093805	0.022366	4.1940	0.000027
beta1	0.892446	0.029969	29.7785	0.000000

LogLikelihood : 34139.66

Information Criteria

Akaike	-5.7692
Bayes	-5.7655
Shibata	-5.7692
Hannan-Quinn	-5.7680

Weighted Ljung-Box Test on Standardized Residuals

	statistic	p-value
Lag[1]	4.405	0.0358308
Lag[2*(p+q)+(p+q)-1][5]	11.630	0.0000000
Lag[4*(p+q)+(p+q)-1][9]	13.936	0.0001326
d.o.f=2		
H0 : No serial correlation		

Weighted Ljung-Box Test on Standardized Squared Residuals

	statistic	p-value
Lag[1]	0.9892	0.3199
Lag[2*(p+q)+(p+q)-1][5]	1.3461	0.7776
Lag[4*(p+q)+(p+q)-1][9]	2.3623	0.8576
d.o.f=2		

Weighted ARCH LM Tests

	Statistic	Shape	Scale	P-Value
ARCH Lag[3]	0.02768	0.500	2.000	0.8679
ARCH Lag[5]	0.73945	1.440	1.667	0.8115
ARCH Lag[7]	1.57450	2.315	1.543	0.8063

Nyblom stability test

Joint Statistic: 6.1912

Individual Statistics:

mu	0.09416
ar1	0.28201
ma1	0.29139
omega	2.42984
alpha1	0.25479
beta1	0.20649

Asymptotic Critical Values (10% 5% 1%)

Joint Statistic:	1.49	1.68	2.12
Individual Statistic:	0.35	0.47	0.75

Sign Bias Test

	t-value	prob	sig
Sign Bias	0.2260	0.821213	
Negative Sign Bias	2.7707	0.005602	***
Positive Sign Bias	0.5128	0.608103	
Joint Effect	9.0697	0.028379	**

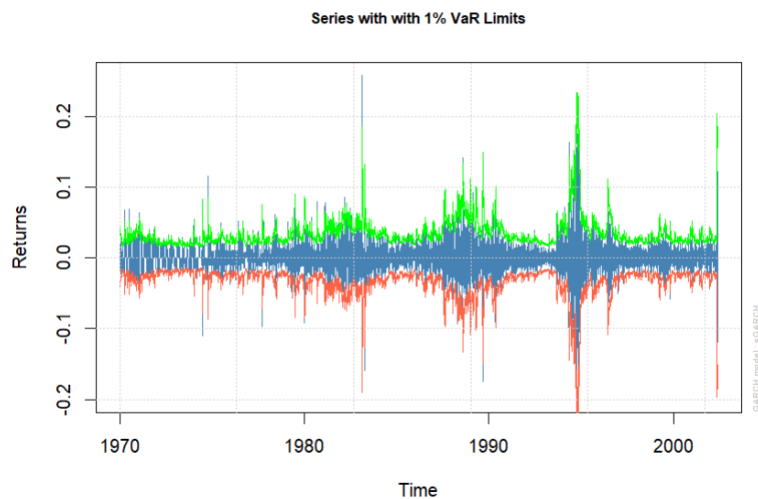
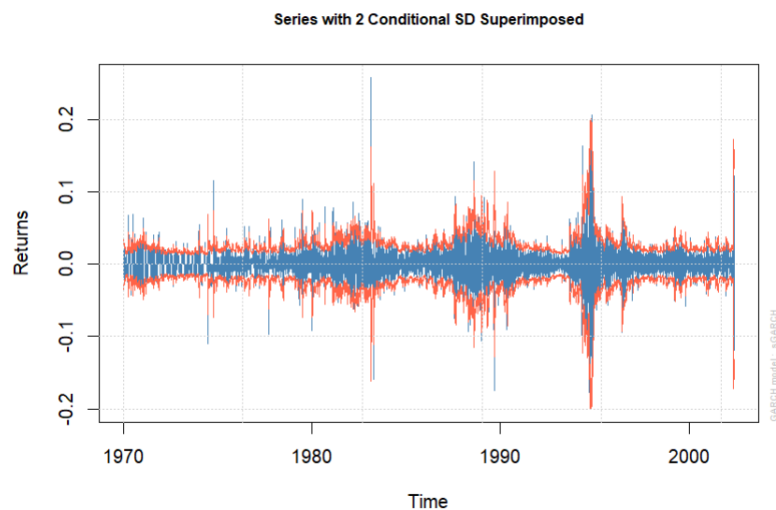
Adjusted Pearson Goodness-of-Fit Test:

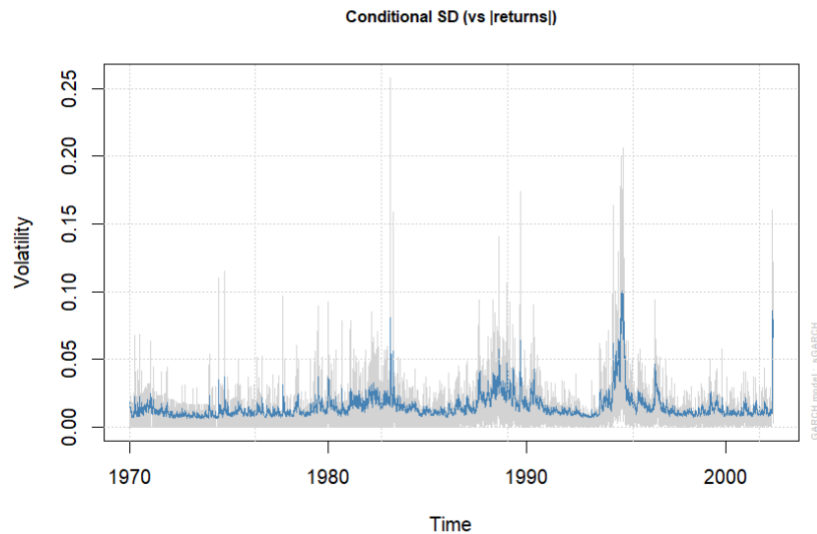
	group	statistic	p-value(g-1)
1	20	10795	0
2	30	14104	0
3	40	10962	0
4	50	15593	0

Elapsed time : 0.983269

> |

These were the residual plots for the returns:





Ljung-Box test was carried out to confirm reliability of the model:

```
resid_std <- residuals(garch_fit, standardize = TRUE)
```

```
Box.test(resid_std, lag = 20, type = "Ljung-Box")
```

```
> Box.test(resid_std, lag = 20, type = "Ljung-Box")
```

Box-Ljung test

data: resid_std

X-squared = 24.126, df = 20, p-value = 0.2369

```
> |
```

p-value > 0.05 verified this. After which, we proceeded to forecasting.

```
# forecast
```

```
fc <- ugarchforecast(garch_fit, n.ahead = 365) # 365-day ahead forecast
fc
```

```
> fc
```

```
*-----*
*      GARCH Model Forecast      *
*-----*
Model: sGARCH
Horizon: 365
Roll Steps: 0
Out of Sample: 0

0-roll forecast [T0=2002-05-26]:
      Series  Sigma
T+1  0.0006422 0.06679
T+2  0.0005743 0.06636
T+3  0.0006231 0.06593
T+4  0.0005880 0.06551
T+5  0.0006132 0.06509
T+6  0.0005951 0.06468
T+7  0.0006081 0.06426
T+8  0.0005988 0.06385
T+9  0.0006055 0.06345
T+10 0.0006007 0.06304
T+11 0.0006041 0.06264
T+12 0.0006016 0.06224
```

```
::
```

```
::
```

```
T+344 0.0006027 0.01858
T+345 0.0006027 0.01856
T+346 0.0006027 0.01855
T+347 0.0006027 0.01854
T+348 0.0006027 0.01852
T+349 0.0006027 0.01851
T+350 0.0006027 0.01850
T+351 0.0006027 0.01849
T+352 0.0006027 0.01847
T+353 0.0006027 0.01846
T+354 0.0006027 0.01845
T+355 0.0006027 0.01844
T+356 0.0006027 0.01843
T+357 0.0006027 0.01842
T+358 0.0006027 0.01840
T+359 0.0006027 0.01839
T+360 0.0006027 0.01838
T+361 0.0006027 0.01837
T+362 0.0006027 0.01836
T+363 0.0006027 0.01835
T+364 0.0006027 0.01834
T+365 0.0006027 0.01833
```

```
> |
```

Then returns were converted back to normal prices to aid forecast visualization.

```
# Convert cumulative log-return forecast back to price|
last_price <- as.numeric(tail(close_ts, 1))

ret_forecast <- as.numeric(fitted(fc))
price_forecast <- last_price * exp(cumsum(ret_forecast))

price_forecast
```

```
> price_forecast
[1] 31.95051 31.96887 31.98879 32.00761 32.02724 32.04630 32.06580 32.08500 32.10444 32.12372
[11] 32.14314 32.16248 32.18189 32.20128 32.22070 32.24012 32.25956 32.27900 32.29847 32.31793
[21] 32.33742 32.35691 32.37642 32.39594 32.41547 32.43501 32.45456 32.47413 32.49370 32.51329
[31] 32.53289 32.55251 32.57213 32.59177 32.61141 32.63107 32.65075 32.67043 32.69012 32.70983
[41] 32.72955 32.74928 32.76902 32.78878 32.80855 32.82833 32.84812 32.86792 32.88773 32.90756
[51] 32.92740 32.94725 32.96711 32.98698 33.00687 33.02677 33.04668 33.06660 33.08653 33.10648
[61] 33.12644 33.14641 33.16639 33.18639 33.20639 33.22641 33.24644 33.26648 33.28654 33.30661
[71] 33.32668 33.34678 33.36688 33.38699 33.40712 33.42726 33.44741 33.46758 33.48775 33.50794
[81] 33.52814 33.54835 33.56858 33.58881 33.60906 33.62932 33.64960 33.66988 33.69018 33.71049
[91] 33.73081 33.75115 33.77149 33.79185 33.81222 33.83261 33.85300 33.87341 33.89383 33.91427
[101] 33.93471 33.95517 33.97564 33.99612 34.01662 34.03712 34.05764 34.07817 34.09872 34.11927
[111] 34.13984 34.16042 34.18102 34.20162 34.22224 34.24287 34.26352 34.28417 34.30484 34.32552
[121] 34.34621 34.36692 34.38764 34.40837 34.42911 34.44987 34.47063 34.49141 34.51221 34.53301
[131] 34.55383 34.57466 34.59551 34.61636 34.63723 34.65811 34.67900 34.69991 34.72083 34.74176
[141] 34.76271 34.78366 34.80463 34.82561 34.84661 34.86762 34.88863 34.90967 34.93071 34.95177
[151] 34.97284 34.99392 35.01502 35.03613 35.05725 35.07839 35.09953 35.12069 35.14186 35.16305
[161] 35.18425 35.20546 35.22668 35.24792 35.26917 35.29043 35.31170 35.33299 35.35429 35.37561
[171] 35.39693 35.41827 35.43962 35.46099 35.48237 35.50376 35.52516 35.54658 35.56800 35.58945
[181] 35.61090 35.63237 35.65385 35.67534 35.69685 35.71837 35.73990 35.76145 35.78301 35.80458
[191] 35.82617 35.84776 35.86937 35.89100 35.91263 35.93428 35.95595 35.97762 35.99931 36.02101
[201] 36.04273 36.06446 36.08620 36.10795 36.12972 36.15150 36.17330 36.19510 36.21692 36.23876
[211] 36.26060 36.28246 36.30434 36.32622 36.34812 36.37003 36.39196 36.41390 36.43585 36.45782
[221] 36.47979 36.50179 36.52379 36.54581 36.56784 36.58989 36.61194 36.63402 36.65610 36.67820
[231] 36.70031 36.72243 36.74457 36.76672 36.78889 36.81107 36.83326 36.85546 36.87768 36.89991
[241] 36.92216 36.94442 36.96669 36.98897 37.01127 37.03359 37.05591 37.07825 37.10060 37.12297
[251] 37.14535 37.16774 37.19015 37.21257 37.23500 37.25745 37.27991 37.30238 37.32487 37.34737
[261] 37.36989 37.39242 37.41496 37.43751 37.46008 37.48267 37.50526 37.52787 37.55050 37.57313
[271] 37.59578 37.61845 37.64113 37.66382 37.68652 37.70924 37.73198 37.75472 37.77748 37.80026
[281] 37.82305 37.84585 37.86866 37.89149 37.91433 37.93719 37.96006 37.98295 38.00584 38.02876
[291] 38.05168 38.07462 38.09757 38.12054 38.14352 38.16652 38.18952 38.21255 38.23558 38.25863
[301] 38.28170 38.30478 38.32787 38.35097 38.37409 38.39723 38.42038 38.44354 38.46671 38.48990
[311] 38.51311 38.53632 38.55956 38.58280 38.60606 38.62933 38.65262 38.67592 38.69924 38.72257
[321] 38.74591 38.76927 38.79264 38.81603 38.83943 38.86284 38.88627 38.90971 38.93317 38.95664
[331] 38.98013 39.00363 39.02714 39.05067 39.07421 39.09776 39.12133 39.14492 39.16852 39.19213
[341] 39.21576 39.23940 39.26305 39.28672 39.31041 39.33411 39.35782 39.38154 39.40529 39.42904
[351] 39.45281 39.47660 39.50039 39.52421 39.54803 39.57188 39.59573 39.61960 39.64349 39.66738
[361] 39.69130 39.71523 39.73917 39.76313 39.78710
> |
```

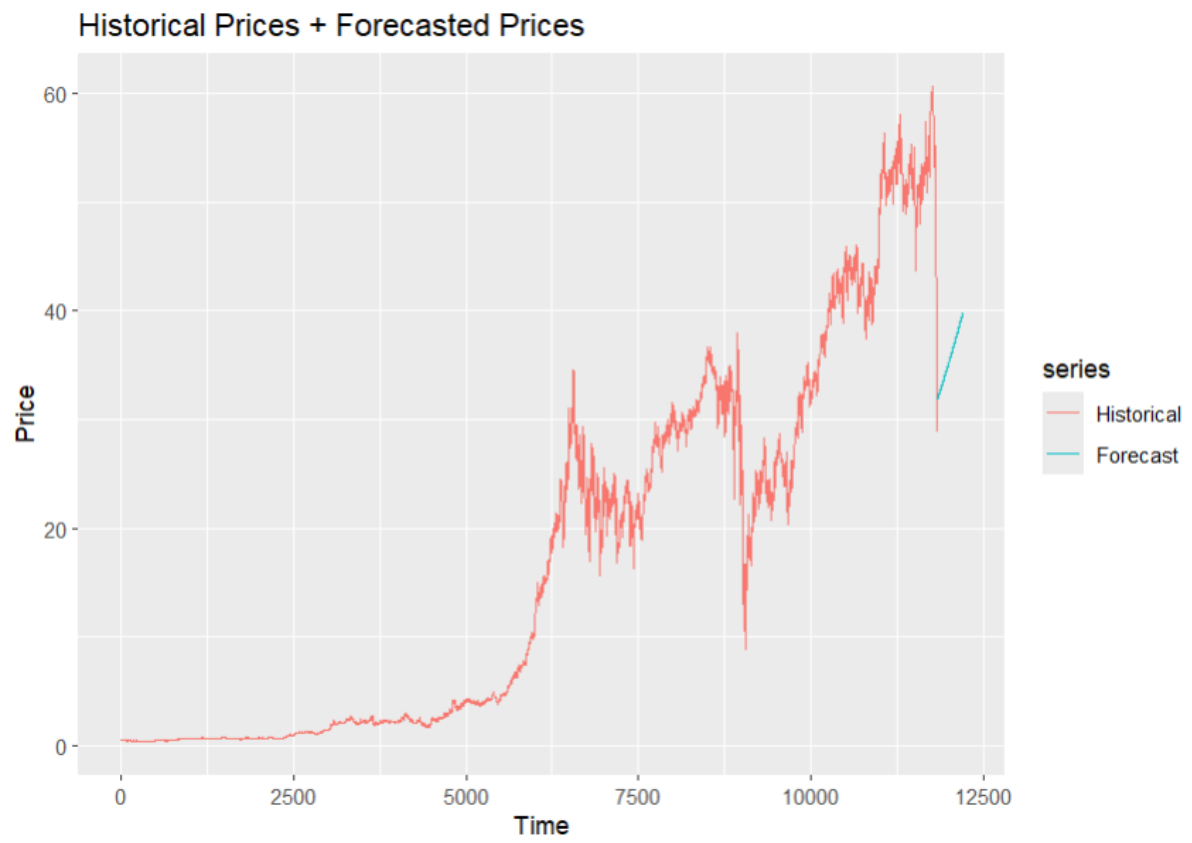
The visualization was then plotted.

```
# convert forecasted prices to a continuation ts|
hist_ts <- ts(usb$Close)

fc_ts <- ts(price_forecast, start = length(hist_ts) + 1)

# combine
merged_ts <- ts.union(Historical = hist_ts, Forecast = fc_ts)

# plot with autoplot
autoplot(merged_ts) +
  ggtitle("Historical Prices + Forecasted Prices") +
  xlab("Time") + ylab("Price")
```

3.3 Conclusion

The US Bancorp (USB) stock showed a clear long-term upward trend but no real seasonality, so trend-based and ARIMA models were the most suitable starting points. Holt's method captured the general direction of the series, but the residual tests showed it wasn't modelling the data well enough. ARIMA improved the fit but still left noticeable autocorrelation, which is common in financial time series where volatility changes over time.

Introducing a GARCH model alongside ARIMA made the biggest difference. This combined approach captured both the average movement of returns and the changing volatility in the market. The diagnostics were stronger, and the resulting price forecasts looked much more realistic compared to the smoother, overly-confident predictions from Holt or standalone ARIMA.

References

Dierckx, T. (2020) *ARIMA-GARCH forecasting with Python*. Medium, 9 September. Available at: <https://medium.com/analytics-vidhya/arima-garch-forecasting-with-python-7a3f797de3ff> (Accessed: 5 December 2025).

Hansanie (n.d.) *Fitting ARIMA–GARCH model using rugarch package*. StackExchange. Available at: <https://stats.stackexchange.com/questions/176550/fitting-arima-garch-model-using-rugarch-package> (Accessed: 5 December 2025).

Navaei, M. (2023) *ARIMA vs GARCH (Time series forecasting)*. LinkedIn. Available at: <https://www.linkedin.com/pulse/arima-vs-garch-time-series-forecasting-mahdi-navaei#:~:text=ARIMA%20models%20are%20best%20suited,and%20other%20data%20quality%20issues>. (Accessed: 5 December 2025).